PSTAT 5LS: Homework 04

Question 5: Are You a Robot?

Your Name Here

Spring 2026

Contents

Background: Are You a Robot? . . . 2
Typesetting in RMarkdown . . . 2
R Markdown Syntax . . . 2
Math Syntax . . . 3
Part 1: Setup . . . 3
Question 1: Fill in the spaces below to correctly write down the setup of this problem. . . . 3
Part 2: Getting Data . . . 3
Question 2: In the space below, type out at least 100 digits (numbers 0 through 9, inclusive), trying to be as random as possible. . . . 3
Question 3: Calculate the number of digits you actually produced, and assign the result to a variable called `n`. . . . 3
Part 3: Analyzing Data . . . 4
Question 4: Calculate the NUMBER of times the number "6" appears in your string of numbers, and assign this to a variable called `num_six`. . . . 4
Question 5: Calculate the PROPORTION of times the number "6" appears in your string of numbers, and assign this to a variable called `prop_six`. . . . 4
Question 6: Suppose we now wish to carry out the hypotheses outlined in the problem statement. Calculate the value of the test statistic, and assign this to a variable called `test_stat`; then print out the value of `test_stat`. . . . 4
Question 7: Calculate the p-value. . . . 4
Question 8: Formulate a conclusion to the test using an $\alpha = 0.05$ significance level. Remember to include: your decision to reject or fail to reject the null hypothesis (and why), whether or not the results are statistically significant, and a clear interpretation of the result in the context of the study . . . 4
Question 9: Check that your answers agree with the results of the `prop_test()` function from the `pstat5lsSBI` package. (This package has already been loaded in for you.) . 5
Part 4: Acknowledging Pitfalls . . . 5
Question 10: Generate an appropriate plot of the distribution of digits you created in Question 2. Think critically about the correct type of plot (yes, there *is* a correct type of plot!) 5
Bonus Question! . . . 5
Closing Instructions . . . 5

VERY IMPORTANT: Remember to start by making a copy of this document, and only edit the copy - **leave this original file untouched** (so your files don't get overwritten every time the server updates). Click here for instructions on how to do that.

Please note that this is only **one question** of Homework 4; the remainder of the questions appear on Gradescope in an "Online Assignment" format. (Please see Canvas for further details.)

Background: Are You a Robot?

Many people like to think they can generate numbers randomly, but evidence shows that humans make pretty bad random number generators. In this problem, we'll work through formally testing how good of a random number generator you are!

Typesetting in RMarkdown

Before we begin with the rest of this problem, note that we'd like to give you some practice formatting R Markdown documents. Recall the following from Lab01:

- *R* is a "statistical computing environment" that's designed for manipulating data, generating plots, and performing analyses. It's also a programming language. You'll be using R this quarter.
- *RStudio* is an "integrated development environment (IDE)" for R (you'll never have to hear the term IDE again in this class). Basically, RStudio is a pretty interface that makes working with R easier. You use R inside of RStudio. If R is ice cream, RStudio is the cone or cup.
- *R Markdown* is a way to write pretty analysis reports that combines R code, R output (plots, analysis results, etc.) and text in one document. This lab document is an R Markdown document!

We've been using `R` periodically throughout this course. `RStudio` is what we've been running on our JupyterHub server. Believe it or not, we've actually had practice with `R Markdown` files as well - that's the filetype of our Post-Labs! (Recall that the filetype of a file is indicated by the suffix in its name; all of our post-labs have filenames of the form `labxx-postlab.RMD` meaning they are `RMD`s: **R** **M**arkdown **D**ocuments).

R Markdown Syntax

R Markdown comes with its own **syntax** (i.e. codes/commands) used to generate the aforementioned "pretty analysis reports." First, some terminology:

- We **knit** an R Markdown Document to turn it into a PDF

You may have noticed that text written in an RMD gets knit directly to text in a PDF - that is, there's no special syntax for getting *text* to appear in a knitted PDF.

Woah, look at that - how did we get the word "text" to appear in italics? This is what we meant by R Mardown "syntax" - there are some special commands/things we need to do in order to get R Studio to correctly knit our RMDs into PDFs. (Is your head swimming with acronyms yet?)

Here's the general workflow. We use special commands in our RMD (some of which we'll discuss in a bit). When R Studio knits our RMD, it looks for any of those special commands, and in turn makes them appear differently in our PDF. This is to say - not all text in our RMD will look the same as it does in the PDF.

For example, to get a word appear in italics, we surround it with single underbraces (_). So, *text* (which appears as `_text_` in the RMD) will look italicized in our final PDF (take a look at the final knitted PDF if you don't believe me!)

Here is a table of some special R Markdown syntax:

Formatting	**Syntax**	**Final Look**
Boldface	`**text**` or `__text__`	**text**
Italics	`_text_` or `*text*`	*text*
Superscript	`^text^`	^{text}

For example, take a look at how the following sentence appears in the RMD and how it appears in the PDF:

> The **quick** brown *fox* jumped over the laz^{y} dog.

Math Syntax

Another neat thing about R Markdown is its ability to typeset equations and mathematical symbols. If you're curious, it does so using yet another programming language called LaTeX; we won't deal much with LaTeX in this course, but one thing you should know is that equations must be surrounded by dollar signs ($) on either side, with **no spaces between the dollar signs and your equations.** Here's a crash course on some syntax for math in R Markdown:

- `\neq` typsets as the $\neq$ symbol: like $p \neq 0.54$ (written as `$p \neq 0.54$`)
- Underscores (`_`) typeset as subscripts, like H_0 (written as `$H_0$`)
- `\hat{p}` gives the $\hat{p}$ symbol: like $\hat{p} = 0.22$ (written as `$\hat{p} = 0.54$`)

Again, it's very important that there isn't a space between either of your dollar signs and your equations. For example, `$ p \neq 0.24$` would NOT typeset correctly, because of the space between the initial dollar sign and the start of the equation.

We expect the remainder of this RMD to typeset correctly, including any/all equation formatting!

Part 1: Setup

The main idea behind this lab is: you'll try to generate some random numbers by hand. Then, we'll conduct a formal statistical test to test the hypothesis that your numbers were truly random against the alternative that they were not (i.e. a two-sided alternative). More specifically, what we'll do is look at the proportion of times the number "6" appears in your string of numbers, and test whether this is consistent with what we would expect under pure chance (indicating that your numbers were randomly-generated) or not (indicating that your numbers were not randomly-generated).

Question 1: Fill in the spaces below to correctly write down the setup of this problem.

- **The parameter is:** *write the definition of your parameter here, using proper mathematical notation!*
- **The null hypothesis is:** *write the null hypothesis here, USING CORRECT LATEX SYNTAX*
- **The alternative hypothesis is:** *write the alternative hypothesis here, USING CORRECT LATEX SYNTAX*

Part 2: Getting Data

It's time for some data!

Question 2: In the space below, type out at least 100 digits (numbers 0 through 9, inclusive), trying to be as random as possible.

```
my_numbers <- ""
## add your numbers between the two quotation marks
```

You do not need to understand what this next code chunk is doing (but, if you're curious, it's splitting up your numbers into distinct digits)

```
my_numbers_split <- strsplit(my_numbers,
                             split = "") |> unlist() |> as.numeric()
```

Question 3: Calculate the number of digits you actually produced, and assign the result to a variable called `n`.

```
## REPLACE THIS COMMENT WITH YOUR CODE; MAKE SURE TO DELETE THE DOUBLE HASHTAGS
```

Hint: recall one of the functions introduced in a previous lab.

If your value of `n` is less than 100, please re-do Question 2 to get at least 100 digits.

Part 3: Analyzing Data

Question 4: Calculate the NUMBER of times the number "6" appears in your string of numbers, and assign this to a variable called `num_six`.

```
## REPLACE THIS COMMENT WITH YOUR CODE; MAKE SURE TO DELETE THE DOUBLE HASHTAGS
```

Hint: Recall the `R` functions you played around with in Lab03. Furthermore, recall that the variable `my_numbers_split` contains a "separated" version of the numbers you created.

Question 5: Calculate the PROPORTION of times the number "6" appears in your string of numbers, and assign this to a variable called `prop_six`.

```
## REPLACE THIS COMMENT WITH YOUR CODE; MAKE SURE TO DELETE THE DOUBLE HASHTAGS
```

Hint: Try and use previously-defined variables as much as possible.

Question 6: Suppose we now wish to carry out the hypotheses outlined in the problem statement. Calculate the value of the test statistic, and assign this to a variable called `test_stat`; then print out the value of `test_stat`.

```
## REPLACE THIS COMMENT WITH YOUR CODE; MAKE SURE TO DELETE THE DOUBLE HASHTAGS
```

Hint: `sqrt()` is the `R` function used to calculate the square root.

Question 7: Calculate the p-value.

```
## REPLACE THIS COMMENT WITH YOUR CODE; MAKE SURE TO DELETE THE DOUBLE HASHTAGS
```

Hint: If necessary, you can use the `abs()` function in `R` to calculate the absolute value of a number.

Question 8: Formulate a conclusion to the test using an $\alpha = 0.05$ significance level. Remember to include: your decision to reject or fail to reject the null hypothesis (and why), whether or not the results are statistically significant, and a clear interpretation of the result in the context of the study

Replace this line with your answers

Question 9: Check that your answers agree with the results of the `prop_test()` function from the `pstat5lsSBI` package. (This package has already been loaded in for you.)

```
## REPLACE THIS COMMENT WITH YOUR CODE; MAKE SURE TO DELETE THE DOUBLE HASHTAGS
```

Part 4: Acknowledging Pitfalls

Though we have just conducted a perfectly sound analysis, it is based on a bit of a simplification. Namely, we have really only tested whether or not the proportion of *sixes* in our data is consistent with what we would expect if our numbers were randomly generated - we completely ignored all the other digits that appeared in our data!

There exist more sophisticated statistical analyses to carry out a more sophisticated test of our original question, however we'll suffice with a simple **statistical visualization.**

Question 10: Generate an appropriate plot of the distribution of digits you created in Question 2. Think critically about the correct type of plot (yes, there *is* a correct type of plot!)

```
## REPLACE THIS COMMENT WITH YOUR CODE; MAKE SURE TO DELETE THE DOUBLE HASHTAGS
```

Based on this plot, do you think your numbers were truly randomly generated? What would your plot look like if your numbers *were* truly randomly generated?

Replace this line with your answers

Bonus Question!

We asked you to ensure you generated at least 100 observations. Why do you think that is - what is special about the number 100 **in the context of this problem**? (There is one very specific answer we are looking for; any deviations from that answer will not receive credit.)

Replace this line with your answers.

Closing Instructions

Congratulations on finishing! When you are done, please do the following:

1) Click the "Knit" button at the top of your RStudio screen (it should have an icon that looks like a ball of yarn). This will create a PDF out of your work.
2) Check that all formatting appears correctly, and that all equations are rendering properly. If not, go back and fix any issues.
3) Download your PDF.
4) Upload your PDF to the appropriate part of the Homework 4 assignment in Gradescope.